

Kubernetes-05

1. Create a namespace dev-environment and apply a resource-based quota that restricts the number of pods to 3 and services to 2.

Create Namespace:

```
root@master:~# kubectl create namespace dev-environment
namespace/dev-environment created
```

resourcequota.yaml:

```
root@master:~# vi resourcequota.yaml
root@master:~# cat resourcequota.yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: dev-quota
  namespace: dev-environment
spec:
  hard:
    pods: "3"
    services: "2"
```

Apply:

```
root@master:~# kubectl apply -f resourcequota.yaml
resourcequota/dev-quota created
```

Verify:

```
root@master:~# kubectl get resourcequota -n dev-environment
NAME          REQUEST          LIMIT          AGE
dev-quota     pods: 0/3, services: 0/2          12s
root@master:~# kubectl describe resourcequota dev-quota -n dev-environment
Name:         dev-quota
Namespace:    dev-environment
Resource      Used  Hard
-----
pods          0     3
services      0     2
root@master:~#
```

2. Create a pod in the prod-environment namespace with 0.2 CPU and 200Mi memory requests, and 0.5 CPU and 500Mi memory limits.

Create namespace:

```
root@master:~# kubectl create namespace prod-environment
namespace/prod-environment created
```

Create firstpod.yaml:

```
root@master:~# vi firstpod.yaml
root@master:~# cat firstpod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-prod
  namespace: prod-environment
spec:
  containers:
  - name: nginx
    image: nginx

    resources:
      requests:
        cpu: "200m"
        memory: "200Mi"

      limits:
        cpu: "500m"
        memory: "500Mi"
```

Apply:

```
root@master:~# kubectl apply -f firstpod.yaml
pod/nginx-prod created
```

Verify:

```
root@master:~# kubectl describe pod nginx-prod -n prod-environment
Name:          nginx-prod
Namespace:     prod-environment
Priority:       0
Service Account: default
Node:          worknode-2/172.31.39.138
Start Time:    Fri, 03 Jul 2026 10:16:17 +0000
Labels:        <none>
Annotations:   <none>
Status:        Running
IP:            10.244.2.59
IPs:
  IP: 10.244.2.59
Containers:
  nginx:
    Container ID:  containerd://09c2680cfd4edad284d38be0c1bc318342be35848e6f680046ae6e450642b314
    Image:         nginx
    Image ID:      docker.io/library/nginx@sha256:ec4ed8b5299e5e90694af7750eb6dffd2627317d30544d056b0371f8082f7bce
    Port:         <none>
    Host Port:    <none>
    State:        Running
    Started:      Fri, 03 Jul 2026 10:16:19 +0000
```

```
Containers:
  nginx:
    Container ID:   containerd://09c2680cfd4edad284d38be0c1bc318342be35848e6f680046ae6e450642b314
    Image:          nginx
    Image ID:       docker.io/library/nginx@sha256:ec4ed8b5299e5e90694af7750eb6dffd2627317d30544d056b0371f8082f7bce
    Port:          <none>
    Host Port:     <none>
    State:         Running
      Started:     Fri, 03 Jul 2026 10:16:19 +0000
    Ready:         True
    Restart Count:  0
    Limits:
      cpu:         500m
      memory:      500Mi
    Requests:
      cpu:         200m
      memory:      200Mi
    Environment:   <none>
```

3. In the staging-environment namespace, set a LimitRange that assigns default CPU and memory limits (300m CPU, 600Mi memory) and applies a minimum and maximum CPU.

Create namespace:

```
root@master:~# kubectl create namespace staging-environment
namespace/staging-environment created
```

Create limitrange.yaml

```
root@master:~# vi limitrange.yaml
root@master:~# cat limitrange.yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: staging-limit
  namespace: staging-environment
spec:
  limits:
  - type: Container

    default:
      cpu: "300m"
      memory: "600Mi"

    defaultRequest:
      cpu: "100m"
      memory: "200Mi"

    min:
      cpu: "100m"

    max:
      cpu: "500m"
```

Apply:

```
root@master:~# kubectl apply -f limitrange.yaml
limitrange/staging-limit created
```

Verify:

```
root@master:~# kubectl describe limitrange staging-limit -n staging-environment
Name:          staging-limit
Namespace:     staging-environment
Type           Resource  Min    Max    Default Request  Default Limit  Max Limit/Request Ratio
-----
Container      cpu        100m   500m   100m           300m           -
Container      memory     -      -      200Mi          600Mi          -
```

Create firstpod.yaml file

```
root@master:~# cat firstpod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  namespace: staging-environment
spec:
  containers:
  - name: nginx
    image: nginx
root@master:~#
```

Apply:

```
root@master:~# kubectl apply -f firstpod.yaml
pod/nginx created
```

Verify:

```
root@master:~# kubectl describe pod nginx -n staging-environment
Name:          nginx
Namespace:     staging-environment
Priority:       0
Service Account: default
Node:          worknode-1/172.31.34.163
Start Time:    Fri, 03 Jul 2026 10:23:35 +0000
Labels:        <none>
Annotations:   kubernetes.io/limit-ranger: LimitRanger plugin set: cpu, mem
Status:        Running
IP:            10.244.1.29
IPs:
  IP: 10.244.1.29
```

Container creates with in the range:

```
containers:
  nginx:
    Container ID:   containerd://8ddf1dc062938fae1bb12c7a376e0bccf0b5fc553750972674279dbac60a40c0
    Image:          nginx
    Image ID:       docker.io/library/nginx@sha256:ec4ed8b5299e5e90694af7750eb6dffd2627317d30544d056b0371f8082f7bce
    Port:           <none>
    Host Port:      <none>
    State:          Running
      Started:      Fri, 03 Jul 2026 10:23:37 +0000
    Ready:          True
    Restart Count:  0
    Limits:
      cpu:          300m
      memory:       600Mi
    Requests:
      cpu:          100m
      memory:       200Mi
    Environment:    <none>
```

4. Create a pod and a NodePort service in the default namespace, then create another pod in the test namespace and communicate between them using Service DNS.

Create pod:

```
root@master:~# cat firstpod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  containers:
  - name: nginx
    image: nginx
```

Apply:

```
root@master:~# kubectl apply -f firstpod.yaml
pod/nginx created
root@master:~# kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
nginx     1/1     Running   0           3s
```

Create nordport service:

```
root@master:~# cat service.yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: NodePort

  selector:
    app: nginx

  ports:
    - port: 80
      targetPort: 80
```

Apply:

```
root@master:~# kubectl apply -f service.yaml
service/nginx-service created
```

Create namespace:

```
root@master:~# kubectl create namespace test
namespace/test created
```

Create busy box pod with namespace test:

```
root@master:~# vi busybox.yaml
root@master:~# cat busybox.yaml
apiVersion: v1
kind: Pod
metadata:
  name: busybox
  namespace: test
spec:
  containers:
    - name: busybox
      image: busybox
      command: ["sleep","3600"]
```

Apply:

```
root@master:~# kubectl apply -f busybox.yaml
pod/busybox created
```

Connect to the pod:

```
root@master:~# kubectl exec -it busybox -n test -- sh
/ # nslookup nginx-service.default.svc.cluster.local
Server:      10.96.0.10
Address:     10.96.0.10:53

Name:   nginx-service.default.svc.cluster.local
Address: 10.97.190.34
```

Inside busybox:

```
/ # wget -qO- http://nginx-service.default.svc.cluster.local
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
/ #
```

5. Apply a LimitRange with a max limit/request ratio of 2 for memory in the performance-environment namespace, and test by creating a pod with mismatched resource requests and limits.

Create namespace:

```
root@master:~# kubectl create namespace performance-environment
namespace/performance-environment created
```

Create limitratio:

```
root@master:~# cat limitratio.yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: ratio-limit
  namespace: performance-environment

spec:
  limits:
  - type: Container

  maxLimitRequestRatio:
    memory: "2"
```

Apply:

```
root@master:~# kubectl apply -f limitratio.yaml
limitrange/ratio-limit created
```

Create pod with resource limit:

```
root@master:~# vi firstpod.yaml
root@master:~# cat firstpod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: test-pod
  namespace: performance-environment

spec:
  containers:
  - name: nginx
    image: nginx

    resources:
      requests:
        memory: "200Mi"

      limits:
        memory: "600Mi"
```

Apply:

```
root@master:~# kubectl delete pods --all
pod "nginx" deleted from default namespace
root@master:~# kubectl apply -f firstpod.yaml
Error from server (Forbidden): error when creating "firstpod.yaml": pods "test-pod" is forbidden: memory max limit to request ratio per Container is 2, but provided ratio is 3.000000
```

Through the error while creating because we are set the limit ratio is 2
(600/200=3)

Now change the resource limit ($600/300=2$)

Create pod:

```
root@master:~# cat firstpod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: test-pod
  namespace: performance-environment
spec:
  containers:
  - name: nginx
    image: nginx
    resources:
      requests:
        memory: "300Mi"
      limits:
        memory: "600Mi"
root@master:~#
```

Apply:

```
root@master:~# kubectl apply -f firstpod.yaml
pod/test-pod created
```

Now it is created